Rust: Segurança na Programação

Enums e Matching

Nuno Neto, 2026

CONNECT5
EDIH European Digital Innovation Hubs Network
REPÚBLICA PORTUGUESA
Financiado pela União Europeia NextGenerationEU
PRR Plano de Recuperação e Resiliência

O que são Enums?

Um **enum** (*enumeration*) é um tipo de dados que define um conjunto de constantes **nomeadas**.

Melhoram a **legibilidade** e **segurança** do código ao restringir os valores possíveis a um conjunto definido de variantes simbólicas.

C

Alias de inteiro, simples e direto

C++

Enum class fortemente tipada

Java

Objetos completos na heap (singletons)

Rust

Tipos de dados algébricos (Sum Types)

Comparação de Enums por Linguagem

Feature	C	C++	Java	Rust
Segurança de tipos	Não	Sim	Sim	Sim
Namespace	Global	Scoped	Scoped	Scoped
Representação em memória	Inteiro	Inteiro mínimo	Objeto na heap	Union nomeada
Dados associados	N/A	N/A	Sim (Singleton)	Sim, sem restrições
Pattern matching	Não	Switch (limitado)	Switch (limitado)	Match e if let

Enums em Rust: Sum Types

Em Rust, enums são **tipos de dados algébricos (TDA)** — especificamente **Sum Types**.

- Podem conter **dados associados**
- Variantes podem ter **tipos diferentes de dados**
- **Structs** são *product types* (composição de campos)
- **Enums** são *sum types* (uma de várias variantes)

Inspirados em linguagens funcionais, são mais expressivos que qualquer implementação anterior.

Definir Enums em Rust

Cada **variante** pode conter um tuplo, uma estrutura semelhante a structs, ou nenhum campo associado.

Variante Simples

ExemploSimples — sem dados associados

Variante Tuplo

`ExemploTupulo(u32, u32)` — dados posicionais

Variante Struct

`ExemploStruct { altura: u32, largura: u32 }` — campos nomeados

Uma instância de um enum pode ser **apenas uma** das variantes em qualquer momento — nunca mais do que uma.

Sum Types na Prática: Formas Geométricas

Sem Sum Types ❌

Uma única struct com campos opcionais para cada forma — campos indefinidos dependendo da forma representada. Difícil e inseguro.

```
struct Forma {
  q_lado: f64,
  r_altura: f64,
  r_largura: f64,
  t_base: f64,
  raio: f64,
  forma: Formas
}

enum Formas {
  Quadrado,
  Rectangulo,
  Triangulo,
  Circulo
}
```

Com Sum Types ✅

Cada variante contém apenas os campos relevantes. Limpo, seguro e expressivo.

```
enum Forma {
 Quadrado { lado: f64 },
 Rectangulo {
 altura: f64,
 largura: f64
 },
 Triangulo {
 base: f64,
 altura: f64
 },
 Circulo { raio: f64 }
}
```

Aceder a um Enum: match e if let

match

Operador de controlo de fluxo poderoso. Compara um valor a uma série de padrões. **Todas as possibilidades têm de estar cobertas.**

if let

Combinação de if com let para **atribuição condicional**. Menos verboso quando apenas um ramo importa.

O Operador match: Padrões Avançados

1

Concatenação de padrões

Usar | para agrupar múltiplos padrões num mesmo braço.

2

Operador ..

Ignorar campos não relevantes do enum ou struct.

3

Ranges e Binding (@)

Enquadrar valores em intervalos e fazer bind da variável com @.

4

Match Guards

Predicados condicionais com if embutido no padrão.

5

Literais e Structs

Corresponder a valores literais, strings, inteiros e structs completos.

O match não se aplica apenas a enums — funciona com Strings, inteiros, flutuantes, structs e muito mais.

Matches e Ownership

Se um valor contém objetos não **Copy**, o match interage com a **ownership**. Tal como nos iteradores, existem três opções:

Referências são pushed down: ao fazer match por referência a um enum, obtém-se acesso por referência aos seus membros — não a ownership do valor interior.

O padrão _ não faz bind e não move o valor. O padrão _str faz bind mas move o valor.

Match por ownership

Move o valor — não utilizável depois

Match por referência (&)

Valor continua utilizável após o match

Match por referência mutável (&mut)

Permite modificar o valor durante o match

Exemplos de match: Formas Geométricas

Match com ownership (cálculo de área):

```
fn calcular_area(forma: Forma) -> f64 {
  match forma {
    Forma::Quadrado { lado } => lado * lado,
    Forma::Rectangulo { altura, largura } => altura *
largura,
    Forma::Triangulo { base, altura } => (base * altura)
/ 2.0,
    Forma::Circulo { raio } => 3.14159 * raio * raio
  }
}
```

Match por referência (descrição):

```
fn descrever(forma: &Forma) -> &str {
  match forma {
    Forma::Quadrado { .. } => "Um quadrado",
    Forma::Rectangulo { altura, largura }
      if altura == largura => "Um quadrado disfarçado",
    Forma::Rectangulo { .. } => "Um rectângulo",
    Forma::Triangulo { .. } => "Um triângulo",
    Forma::Circulo { raio } if *raio > 10 => "Um círculo
grande",
    Forma::Circulo { .. } => "Um círculo pequeno"
  }
}
```

O primeiro exemplo move a forma (ownership), enquanto o segundo usa referência (&) para preservar o valor original.

O Operador if let

O **if let** é uma forma concisa de lidar com **apenas uma variante**, ignorando as restantes.

Suporta muitos dos padrões do match, incluindo **ranges** e o operador **..**

```
fn raio(&self) -> u32 {
  if let Forma::Circulo {
    raio: &u32
  } = self {
    *raio
  } else { 0 }
}

fn tamanho_base(&self) -> &str {
  if let Forma::Triangulo {
    base: 0..<10, ..
  } = self {
    "pequeno"
  } else { "grande" }
}
```

match

Exaustivo, cobre todas as variantes. Ideal para lógica complexa.

if let

Conciso, cobre uma variante. Ideal para casos simples.

Use **match** quando precisar de cobrir múltiplas variantes.
Use **if let** quando apenas uma variante importa.